Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»
Пермский институт (филиал) РЭУ им. Г.В. Плеханова

Сопроводительная Записка

МДК _____ / ОП ____ «_____»

Специальность _____ «_____»

Тема: Подсистема работы с партнерами на языке C# с использованием технологий WPF и СУБД PostgreSQL

Листов: 53

Руководитель
_____ /Д.Б. Берестов/
«_____» _____ 2026 г.

Исполнитель
_____ /Д.А. Шумиловских/ «_____»
_____ 2026 г.

2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 4
РАЗДЕЛ 1. СОЗДАНИЕ БАЗЫ ДАННЫХ 5
1.1. Создание базы данных и схемы данных 5
1.2. Описание таблиц базы данных 5
1.2.1. Таблица partner_types (Типы партнеров) 6
1.2.2. Таблица partners (Партнеры) 6
1.2.3. Таблица products (Продукция) 7
1.2.4. Таблица sales_history (История продаж) 8
1.3. Связи и целостность данных 9
1.4. Интеграция базы данных с приложением 9
1.5 Схема базы данных 10
РАЗДЕЛ 2. СОЗДАНИЕ БИБЛИОТЕКИ КЛАССОВ 11
2.1. Структура библиотеки классов 11
2.2. Модели данных (Models) 11
2.2.1. Класс PartnerType (Тип партнера) 11
2.2.2. Класс Partner (Партнер) 12
2.2.3. Класс Product (Продукция) 13
2.2.4. Класс SaleHistory (История продаж) 13
2.3. Контекст базы данных (ApplicationContext) 14
2.4. Сервисный слой (Services) 15
2.4.1. Интерфейсы сервисов 15
2.4.2. Класс PartnerService 16
2.4.3. Класс DiscountService 17
2.4.4. Класс ProductService 18
2.4.5. Класс SaleHistoryService 18
2.5. Обработка исключений 19
2.6. Внедрение зависимостей 19
2.7 Диаграмма классов 20
РАЗДЕЛ 3. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ 21
3.1. Инструменты и среда тестирования 21
3.2. Сценарии тестирования 21

3.3. Результаты тестирования 22
РАЗДЕЛ 4. СОЗДАНИЕ ПРОЕКТА WPF 24
4.1. Общая структура приложения 24
4.2. Главное окно MainWindow 24
4.2.1 Интерфейс окна MainWindow 24
4.2.2 Логика работы окна MainWindow 27
4.3. Окно управления партнерами PartnersWindow 30
4.3.1 Интерфейс PartnersWindow 30
4.3.2 Логика работы окна PartnersWindow 32
4.4 Окно управления продукцией ProductsWindow 37
4.4.1. Интерфейс ProductsWindow 37
4.4.2 Логика работы окна ProductsWindow 38
4.5 Окно добавления и изменения продукции ProductEditWindow 39
4.5.1 Интерфейс ProductEditWindow 39
4.5.2 Логика работы окна ProductEditWindow 40
4.6 Окно истории продаж SalesHistoryWindow 42
4.6.1 Интерфейс SalesHistoryWindow 42
4.6.2 Логика работы окна SalesHistoryWindow 44
4.7 Окно добавления продажи SaleEditWindow 47
4.7.1 Интерфейс SaleEditWindow 47
4.7.2 Логика работы окна SaleEditWindow 48
ЗАКЛЮЧЕНИЕ 52
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ 53

ВВЕДЕНИЕ

В ходе выполнения проекта мной было разработано desktop-приложение «Подсистема работы с партнерами» (проект shumilovskih_pp). Основная идея проекта заключалась в создании удобной программы для автоматизации учета партнеров компании, их продукции и истории продаж. Раньше подобные данные могли вестись вручную или в разрозненных файлах, что неудобно и ведет к ошибкам. Мое приложение позволяет в одном месте хранить информацию о контрагентах, вести каталог товаров и фиксировать каждую продажу. Также в проекте реализована автоматическая система скидок: программа сама считает общую сумму покупок партнера и назначает ему уровень скидки (Базовый, Бронзовый, Серебряный или Золотой) в зависимости от оборота.

Для реализации базы данных и самого приложения я использовал следующие инструменты и технологии:

– Microsoft Visual Studio 2022 — среда разработки, в которой я писал код, создавал окна интерфейса и отлаживал программу;

– язык программирования C# 7.3 — основной язык, на котором написана вся логика приложения и работа с данными;

– технология WPF (Windows Presentation Foundation) — использовалась для создания графического интерфейса программы (окна, кнопки, таблицы) с помощью XAML;

– СУБД PostgreSQL 16 — система управления базами данных, где хранятся все таблицы с партнерами, товарами и продажами;

– Entity Framework Core — технология для работы с базой данных из кода C#, чтобы не писать сложные SQL-запросы вручную;

– провайдер Npgsql — драйвер, который позволяет приложению на .NET подключаться к базе данных PostgreSQL;

– DbSchema — инструмент для получения схемы базы данных shumilovskih_pp.

РАЗДЕЛ 1. СОЗДАНИЕ БАЗЫ ДАННЫХ

1.1. Создание базы данных и схемы данных

Для хранения информации о партнерах, продукции и историях продаж мной была выбрана реляционная СУБД PostgreSQL 16. Это решение обусловлено её надежностью, поддержкой сложных типов данных и отличным взаимодействием с платформой .NET через провайдера Npgsql.

Процесс создания базы данных я выполнял последовательно, используя SQL-скрипты. Сначала создается сама база данных, затем внутри неё выделяется отдельная схема app для логического группирования таблиц. Это позволяет отделить объекты приложения от системных объектов PostgreSQL.

Последовательность команд для создания структуры выглядит следующим образом:

– создание роли app:

```
create role app with password ‘123456789’;
```

– создание базы данных:

```
create database shumilovskih_pp;
```

– подключение к базе данных:

```
\c shumilovskih_pp
```

– создание схемы приложения:

```
CREATE SCHEMA app;
```

– настройка прав доступа:

Все таблицы и последовательности создаются владельцем app, что обеспечивает корректное управление правами доступа в дальнейшем [1].

```
shumilovskih_pp=# \dt
                Список отношений
 Схема |      Имя      |   Тип   | Владелец
-------+---------------+---------+----------
 app   | partner_types | таблица | app
 app   | partners      | таблица | app
 app   | products      | таблица | app
 app   | sales_history | таблица | app
(4 строки)
```

Рис. 1 — **список отношений бд shumilovskih_pp**

1.2. Описание таблиц базы данных

В рамках проекта было спроектировано четыре основные таблицы. Каждая таблица имеет первичный ключ (Primary Key), а также необходимые ограничения (Constraints) для обеспечения целостности данных. Ниже приведено подробное описание структуры каждой таблицы.

1.2.1. Таблица partner_types (Типы партнеров)

Это справочная таблица. Она нужна, чтобы классифицировать партнеров (например, «Строительный магазин» или «Оптовый склад») [2].

Поля таблицы partner_types:

– partner_type_id (integer) – уникальный идентификатор типа. Это первичный ключ (PRIMARY KEY), значение заполняется автоматически через последовательность.

– type_name (character varying(100)) – название типа партнера. Поле обязательное (NOT NULL) и должно быть уникальным (UNIQUE), чтобы не было дублей.

```
shumilovskih_pp=# select * from partner_types;
 partner_type_id |      type_name
-----------------+---------------------
               1 | Строительный магазин
               2 | Оптовый склад
               3 | Дистрибьютор
               4 | Интернет-магазин
               5 | Розничная сеть
               6 | 4545
(6 строк)
```

Рис. 2 — **таблица partner_types**

1.2.2. Таблица partners (Партнеры)

Основная таблица, где хранятся реквизиты и контакты компаний [3].

Поля таблицы partners:

– partner_id (integer) — уникальный идентификатор партнера. Первичный ключ, автоинкремент.

– company_name (character varying(255)) — название компании. Обязательное поле.

– partner_type_id (integer) — ссылка на тип партнера. Внешний ключ (FOREIGN KEY) на таблицу partner_types.

– inn (character varying(20)) — идентификационный номер налогоплательщика.

– logo (character varying(500)) — путь к файлу логотипа компании.

– rating (integer) — рейтинг партнера. Обязательное поле, стоит ограничение CHECK (rating >= 0), чтобы рейтинг не был отрицательным.

– address (character varying(500)) — юридический адрес.

– director_name (character varying(255)) — ФИО директора.

– phone (character varying(50)) — контактный телефон.

– email (character varying(255)) — электронная почта.

– sales_locations (text) — текстовое описание мест продаж.

– created_date (timestamp) — дата создания записи. По умолчанию ставится текущее время.

– modified_date (timestamp) — дата последнего изменения. По умолчанию текущее время.

```
shumilovskih_pp=# select * from partners;
 partner_id | company_name  | partner_type_id |     inn     |                          logo                          | rating |              address              |        director_name        |       phone
            |       email       |      sales_locations      |        created_date        |        modified_date
------------+---------------+-----------------+-------------+--------------------------------------------------------+--------+-----------------------------------+-----------------------------+-------------
-------+-----------------+-----------------+-----------------+-----------------------------+-----------------------------+-----------------------------
          1 | СтройДом      |               1 | 7701123456  | /logos/stroydom.png                                    |      5 | г. Москва, ул. Строителей, 10     | Иванов Иван Иванович        | +7 (495) 123
-45-67 | info@stroydom.ru  | Москва, Московская область | 2026-03-16 00:03:05.854922 | 2026-03-16 00:03:05.854922
          2 | ОптТорг       |               2 | 7701987654  | /logos/opttorg.png                                     |      4 | г. Санкт-Петербург, пр. Промышленный, 5 | Петров Петр Петрович | +7 (812) 987
-65-43 | sales@opttorg.ru  | СЗФО                | 2026-03-16 00:03:05.854922 | 2026-03-16 00:03:05.854922
          3 | Домовой       |               5 | 7702123450  | /logos/domovoy.png                                     |      5 | г. Екатеринбург, ул. Ленина, 20   | Сидоров Сидор Сидорович     | +7 (343) 222
-33-44 | info@domovoy.ru   | Урал                | 2026-03-16 00:03:05.854922 | 2026-03-16 00:03:05.854922
          5 | СтройАрсенал  |               4 | 7704123452  | /logos/stroyarsenal.png                                |      4 | г. Новосибирск, ул. Мира, 30      | Васильев Василий            | +7 (383) 444
-55-66 | shop@stroyarsenal.ru | Сибирь           | 2026-03-16 00:03:05.854922 | 2026-03-16 00:03:05.854922
          6 | Уютный Дом    |               1 | 7705123453  | /logos/uytdom.png                                      |      3 | г. Ростов-на-Дону, ул. Садовая, 12 | Алексеева Анна              | +7 (863) 777
-88-99 | info@uytdom.ru    | ЮФО                 | 2026-03-16 00:03:05.854922 | 2026-03-16 00:03:05.854922
          7 | Ремонт-Сервис |               3 | 7706123454  | /logos/remont.png                                      |      4 | г. Нижний Новгород, ул. Промышленная, 7 | Михайлов Михаил | +7 (831) 333
-44-55 | mih@remont.ru     | ПФО                 | 2026-03-16 00:03:05.854922 | 2026-03-16 00:03:05.854922
          8 | ТД Паркет     |               2 | 7707123455  | /logos/tdparket.png                                    |      5 | г. Воронеж, ул. 20 лет Октября, 50 | Егоров Егор                 | +7 (473) 222
-33-44 | tdparket@mail.ru  | Центр               | 2026-03-16 00:03:05.854922 | 2026-03-16 00:03:05.854922
          9 | Онлайн-Пол    |               4 | 7708123456  | /logos/onlinepol.png                                   |      4 | г. Краснодар, ул. Красная, 1      | Сергеев Сергей              | +7 (861) 555
-77 | info@online-pol.ru  | Интернет                  | 2026-03-16 00:03:05.854922 | 2026-03-16 00:03:05.854922
         10 | Доступный пол |               1 | 7709123457  | /logos/dostupnyi.png                                   |      3 | г. Самара, ул. Дачная, 8          | Федоров Федор               | +7 (846) 444
-55-66 | fedorov@pol.ru    | Самара и область    | 2026-03-16 00:03:05.854922 | 2026-03-16 00:03:05.854922
          4 | Пол-Мастер    |               3 | 7703123451  | /logos/polmaster.png                                   |      5 | г. Казань, ул. Машиностроителей, 15 | Кузнецов Андрей             | +7 (843) 555
-66-77 | zakaz@polmaster.ru | Татарстан, Поволжье | 2026-03-16 00:03:05.854922 | 2026-03-16 00:10:09.483704
         11 | БрухнойПол    |               2 | 454554343   | C:\Users\danil\OneDrive\Рабочий стол\Роор\0153326d7c9c10dc1e51d82a14868bca.jpg |      4 | ул. Уинская 68                    | Колотушкин Дмитрий Алексеевич | +7 (234) 495
 34 34 | ttr@gmail.com     | Где-то              | 2026-03-16 00:14:36.00789  | 2026-03-16 00:16:29.573253
```

Рис. 3 — **таблица partners**

1.2.3. Таблица products (Продукция)

В этой таблице ведется каталог товаров, которые могут продавать партнеры [4].

Поля таблицы products:

– product_id (integer) – уникальный идентификатор товара. Первичный ключ, автоинкремент.

– product_name (character varying(255)) – название продукции. Обязательное поле.

– article (character varying(100)) – артикул товара. Должен быть уникальным (UNIQUE).

– type (character varying(100)) – тип продукции (например, «Ламинат»).

– description (text) – подробное описание товара.

– min_price (numeric(10,2)) – минимальная цена. Обязательное поле, ограничение CHECK (min_price >= 0).

– package_size (character varying(100)) – размер упаковки.

– weight (numeric(10,2)) – вес единицы товара. Ограничение CHECK (weight >= 0).

– standard_number (character varying(100)) – номер стандарта (ГОСТ или EN).

– created_date (timestamp) – дата создания записи.

– modified_date (timestamp) – дата последнего изменения.

```
shumilovskih_pp=# select * from products;
 product_id |       product_name       | article |    type    |             description             | min_price | package_size | weight | standard_number
------------+--------------------------+---------+------------+-------------------------------------+-----------+--------------+--------+-----------------
          1 | Ламинат Дуб белый        | LAM-001 | Ламинат    | Ламинат 33 класс, толщина 8 мм      |    850.00 | 8 шт/упак    |  12.50 | EN 13329
          2 | Ламинат Орех тёмный      | LAM-002 | Ламинат    | Ламинат 33 класс, толщина 8 мм      |    890.00 | 8 шт/упак    |  12.50 | EN 13329
          3 | Паркетная доска Ясень    | PAR-001 | Паркет     | Паркетная доска трёхслойная, 14 мм  |   2100.00 | 4 шт/упак    |  20.00 | EN 14342
          4 | Паркетная доска Дуб      | PAR-002 | Паркет     | Паркетная доска трёхслойная, 14 мм  |   2300.00 | 4 шт/упак    |  20.00 | EN 14342
          5 | Виниловая плитка Серая   | VIN-001 | Винил      | ПВХ плитка замковая                 |    650.00 | 10 шт/упак   |  15.00 | EN 649
          6 | Виниловая плитка Бежевая | VIN-002 | Винил      | ПВХ плитка замковая                 |    670.00 | 10 шт/упак   |  15.00 | EN 649
          7 | Линолеум бытовой         | LIN-001 | Линолеум   | Бытовой линолеум, ширина 2.5 м      |    320.00 | пог. м       |   8.00 | EN 548
          8 | Линолеум коммерческий    | LIN-002 | Линолеум   | Коммерческий линолеум, ширина 2.5 м |    450.00 | пог. м       |   9.50 | EN 548
          9 | Ковровая плитка          | KOV-001 | Ковролин   | Ковровая плитка модульная           |    550.00 | шт           |   2.30 | EN 1307
         10 | Кварцвинил               | KVZ-001 | Кварцвинил | Кварцвиниловая плитка               |    980.00 | шт           |   3.00 | EN 14041
(10 строк)
```

Рис. 4 — **таблица products**

1.2.4. Таблица sales_history (История продаж)

Таблица, где фиксируются все сделки. Здесь видно, кто, что и когда продал [5].

– sale_id (integer) – уникальный идентификатор продажи. Первичный ключ, автоинкремент.

– partner_id (integer) – ссылка на партнера. Внешний ключ на таблицу partners. При удалении партнера записи удаляются каскадом (ON DELETE CASCADE).

– product_id (integer) – ссылка на продукцию. Внешний ключ на таблицу products. При удалении товара удаление запрещается, если есть продажи (ON DELETE RESTRICT).

– quantity (integer) – количество проданного товара. Обязательное поле, ограничение CHECK (quantity > 0).

– sale_date (date) – дата продажи. Обязательное поле, по умолчанию текущая дата.

– total_amount (numeric(12,2)) – общая сумма сделки. Обязательное поле, ограничение CHECK (total_amount >= 0).

created_date (timestamp) – дата создания записи в журнале.

sale_id	partner_id	product_id	quantity	sale_date	total_amount	created_date
1	1	1	10	2025-12-10	8500.00	2026-03-16 00:03:12.132025
2	1	2	5	2025-12-15	4450.00	2026-03-16 00:03:12.132025
3	1	5	8	2026-01-20	5200.00	2026-03-16 00:03:12.132025
4	2	3	12	2025-11-05	25200.00	2026-03-16 00:03:12.132025
5	2	4	6	2025-12-12	13800.00	2026-03-16 00:03:12.132025
6	2	9	15	2026-01-18	8250.00	2026-03-16 00:03:12.132025
7	3	2	20	2025-10-22	17800.00	2026-03-16 00:03:12.132025
8	3	6	10	2025-12-01	6700.00	2026-03-16 00:03:12.132025
9	3	8	30	2026-01-10	13500.00	2026-03-16 00:03:12.132025
10	4	7	25	2025-09-30	8000.00	2026-03-16 00:03:12.132025
11	4	10	12	2025-11-17	11760.00	2026-03-16 00:03:12.132025
12	4	3	4	2026-01-25	8400.00	2026-03-16 00:03:12.132025
13	5	5	15	2025-08-14	9750.00	2026-03-16 00:03:12.132025
14	5	9	8	2025-12-28	4400.00	2026-03-16 00:03:12.132025
15	5	1	10	2026-02-05	8500.00	2026-03-16 00:03:12.132025
16	6	4	5	2025-11-29	11500.00	2026-03-16 00:03:12.132025
17	6	2	7	2026-01-12	6230.00	2026-03-16 00:03:12.132025
18	7	8	20	2025-12-19	9000.00	2026-03-16 00:03:12.132025
19	7	6	9	2026-02-10	6030.00	2026-03-16 00:03:12.132025
20	8	10	8	2025-10-05	7840.00	2026-03-16 00:03:12.132025
21	8	3	10	2025-12-07	21000.00	2026-03-16 00:03:12.132025
22	9	7	15	2025-11-15	4800.00	2026-03-16 00:03:12.132025
23	9	9	5	2026-01-30	2750.00	2026-03-16 00:03:12.132025
24	10	1	8	2025-12-23	6800.00	2026-03-16 00:03:12.132025
25	10	5	12	2026-02-14	7800.00	2026-03-16 00:03:12.132025
26	11	5	12	2026-03-16	221222.00	2026-03-16 00:15:03.703104
27	11	3	34	2026-03-16	1222222.00	2026-03-16 00:15:18.934911

(27 строк)

Рис. 5 — **таблица sales_history**

1.3. Связи и целостность данных

Чтобы данные не терялись и не противоречили друг другу, я настроил внешние ключи. Это связывает таблицы между собой логически.

Партнеры и Типы: Поле partner_type_id в таблице партнеров ссылается на partner_types. Я поставил правило ON DELETE RESTRICT, чтобы нельзя было удалить тип, если к нему привязаны партнеры.

Продажи и Партнеры: Поле partner_id в истории продаж ссылается на партнеров. Здесь стоит ON DELETE CASCADE, потому что если мы удаляем партнера из базы, то и история его продаж становится не нужна.

Продажи и Продукция: Поле product_id ссылается на таблицу продукции. Стоит ON DELETE RESTRICT, чтобы нельзя было удалить товар, который уже числится в проданных.

1.4. Интеграция базы данных с приложением

Для работы с базой данных из кода я использовал технологию Entity Framework Core. Это позволило мне не писать сложные SQL-запросы вручную, а работать с таблицами как с обычными объектами C#.

В проекте есть класс ApplicationContext, который наследуется от DbContext. В нем я описал свойства для каждой таблицы:

```
public DbSet<PartnerType> PartnerTypes { get; set; }
public DbSet<Partner> Partners { get; set; }
public DbSet<Product> Products { get; set; }
public DbSet<SaleHistory> SalesHistory { get; set; }
```

Подключение к базе прописано в строке подключения:

```
Host=localhost;Database=shumilovskih_pp;Username=app;Password=123456789
```

Также в методе OnModelCreating я продублировал настройки схемы и связей, чтобы Entity Framework понимал, как правильно взаимодействовать с таблицами в схеме app.

1.5 Схема базы данных

Для получения схемы базы данных я использовал программу DbSchema [6].

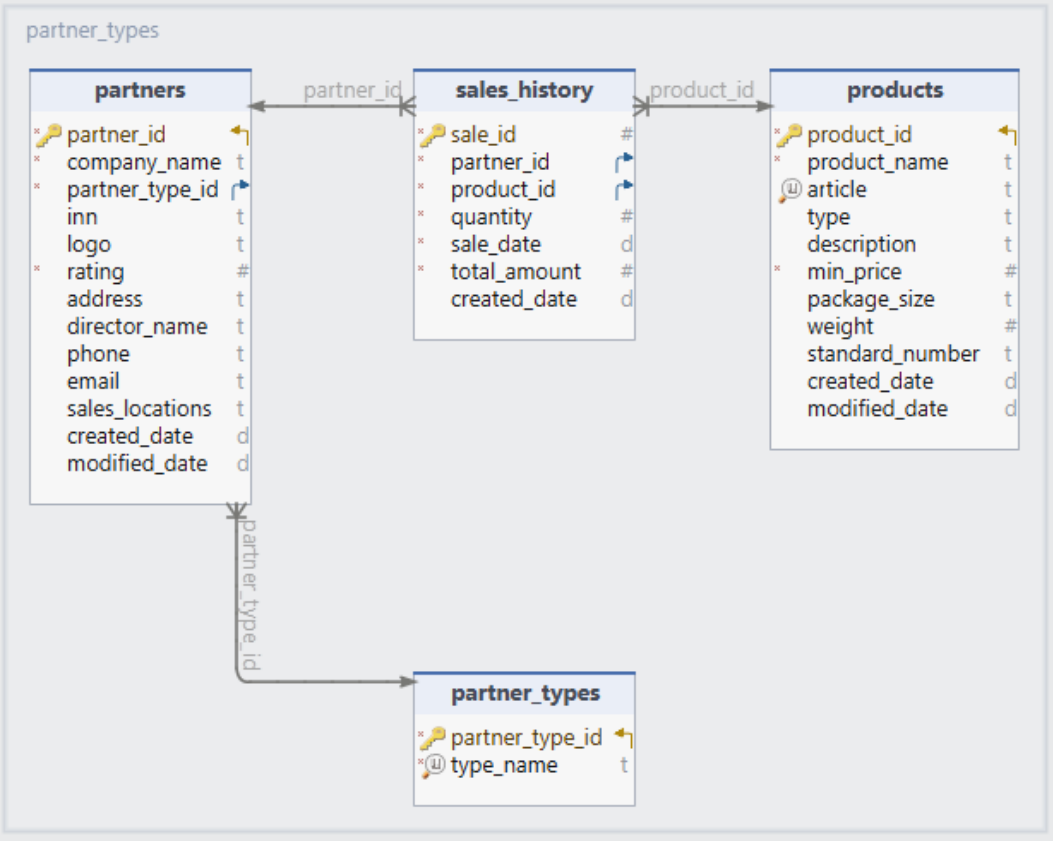

Рис. 6 — **схема базы данных shumilovskih_pp**

РАЗДЕЛ 2. СОЗДАНИЕ БИБЛИОТЕКИ КЛАССОВ

2.1. Структура библиотеки классов

Для реализации бизнес-логики проекта я создал отдельную библиотеку классов shumilovskih_library. Это позволило отделить логику работы с данными от пользовательского интерфейса WPF-приложения. Такая архитектура делает код более понятным, удобным для тестирования и поддержки.

Библиотека имеет следующую структуру папок:

– Context — содержит класс контекста базы данных ApplicationContext для работы с Entity Framework Core;

– Models — классы-модели, соответствующие таблицам базы данных (Partner, Product, SaleHistory, PartnerType);

– Services — сервисные классы с бизнес-логикой и интерфейсы к ним;

– Exceptions — пользовательские исключения для обработки ошибок предметной области.

Ниже я подробно опишу каждый компонент библиотеки.

2.2. Модели данных (Models)

Модели данных представляют собой C#-классы, которые маппятся на таблицы базы данных через Entity Framework Core. Каждый класс имеет атрибуты, указывающие на соответствие таблицам и полям в PostgreSQL.

2.2.1. Класс PartnerType (Тип партнера)

Класс соответствует таблице partner_types и содержит следующие свойства:

– PartnerTypeId (int) — первичный ключ, соответствует полю partner_type_id;

– TypeName (string, макс. 100 символов) — наименование типа партнера, соответствует полю type_name;

– Partners (ICollection<Partner>) — навигационное свойство для связи с таблицей партнеров (один ко многим).

```
[Table("partner_types", Schema = "app")]
```

```
public class PartnerType
{
    [Key]
    [Column("partner_type_id")]
    public int PartnerTypeId { get; set; }
       [Column("type_name")]
    [MaxLength(100)]
    public string TypeName { get; set; } = string.Empty;
        public virtual ICollection<Partner> Partners { get; set; } = new
List<Partner>();
}
```

2.2.2. Класс Partner (Партнер)

Класс соответствует таблице partners и содержит расширенную информацию о компаниях-партнерах:

– PartnerId (int) — первичный ключ, автоинкремент;

– CompanyName (string, макс. 255 символов) — название компании, обязательное поле;

– PartnerTypeId (int) — внешний ключ на тип партнера;

– Inn (string, макс. 20 символов) — ИНН организации;

– Logo (string, макс. 500 символов) — путь к файлу логотипа;

– Rating (int) — рейтинг партнера, должен быть неотрицательным (атрибут [Range]);

– Address (string, макс. 500 символов) — юридический адрес;

– DirectorName (string, макс. 255 символов) — ФИО директора;

– Phone (string, макс. 50 символов) — контактный телефон;

– Email (string, макс. 255 символов) — электронная почта с валидацией формата ([EmailAddress]);

– SalesLocations (string) — информация о местах продаж;

– CreatedDate и ModifiedDate (DateTime) — временные метки создания и изменения;

– PartnerType (PartnerType) — навигационное свойство для связи с типом партнера;

– SalesHistory (ICollection<SaleHistory>) — коллекция истории продаж партнера.

2.2.3. Класс Product (Продукция)

Класс соответствует таблице products и описывает товары в каталоге:

– ProductId (int) — первичный ключ;

– ProductName (string, макс. 255 символов) — название продукции, обязательное поле;

– Article (string, макс. 100 символов) — артикул товара, должен быть уникальным;

– Type (string, макс. 100 символов) — тип продукции;

– Description (string) — подробное описание;

– MinPrice (decimal) — минимальная цена, неотрицательное значение;

– PackageSize (string, макс. 100 символов) — размер упаковки;

– Weight (decimal) — вес единицы товара, неотрицательное значение;

– StandardNumber (string, макс. 100 символов) — номер стандарта (ГОСТ/EN);

– CreatedDate и ModifiedDate (DateTime) — временные метки;

– SalesHistory (ICollection<SaleHistory>) — коллекция истории продаж для данного товара.

2.2.4. Класс SaleHistory (История продаж)

Класс соответствует таблице sales_history и фиксирует транзакции продаж:

– SaleId (int) — первичный ключ;

– PartnerId (int) — внешний ключ на партнера, обязательное поле;

– ProductId (int) — внешний ключ на продукцию, обязательное поле;

– Quantity (int) — количество проданного товара, должно быть больше 0;

– SaleDate (DateTime) — дата продажи, обязательное поле;

– TotalAmount (decimal) — общая сумма сделки, неотрицательное значение;

– CreatedDate (DateTime) — дата создания записи;

– Partner (Partner) — навигационное свойство для связи с партнером;

– Product (Product) — навигационное свойство для связи с продукцией.

2.3. Контекст базы данных (ApplicationContext)

Класс ApplicationContext наследуется от DbContext и служит для взаимодействия с базой данных PostgreSQL через Entity Framework Core. В нём описываются все таблицы как свойства DbSet<T> и настраиваются связи между сущностями.

Для каждой таблицы базы данных я объявил свойство типа DbSet<T>:

```
public DbSet<PartnerType> PartnerTypes { get; set; }
public DbSet<Partner> Partners { get; set; }
public DbSet<Product> Products { get; set; }
public DbSet<SaleHistory> SalesHistory { get; set; }
```

Метод OnConfiguring содержит строку подключения к базе данных PostgreSQL:

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        optionsBuilder.UseNpgsql(

"Host=localhost;Database=shumilovskih_pp;Username=app;Password=123456789"
        );
    }
}
```

Метод OnModelCreating используется для конфигурации связей между таблицами и указания схемы по умолчанию:

– Схема по умолчанию — установлена схема app через modelBuilder.HasDefaultSchema("app");

– Связь Партнеры — Типы партнеров — ON DELETE RESTRICT, чтобы нельзя было удалить тип, если к нему привязаны партнеры;

– Связь История продаж — Партнеры — ON DELETE CASCADE, при удалении партнера удаляются все его продажи;

– Связь История продаж — Продукция — ON DELETE RESTRICT, чтобы нельзя было удалить товар с историей продаж.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.HasDefaultSchema("app");

    modelBuilder.Entity<Partner>()
        .HasOne(p => p.PartnerType)
        .WithMany(pt => pt.Partners)
        .HasForeignKey(p => p.PartnerTypeId)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<SaleHistory>()
        .HasOne(sh => sh.Partner)
        .WithMany(p => p.SalesHistory)
        .HasForeignKey(sh => sh.PartnerId)
        .OnDelete(DeleteBehavior.Cascade);

    modelBuilder.Entity<SaleHistory>()
        .HasOne(sh => sh.Product)
        .WithMany(p => p.SalesHistory)
        .HasForeignKey(sh => sh.ProductId)
        .OnDelete(DeleteBehavior.Restrict);
}
```

2.4. Сервисный слой (Services)

Сервисный слой инкапсулирует бизнес-логику приложения и предоставляет методы для работы с данными. Для каждого типа сущности я создал интерфейс и его реализацию.

2.4.1. Интерфейсы сервисов

Все сервисы имеют асинхронные методы для CRUD-операций:

– IPartnerService — методы для работы с партнерами (GetAllPartnersAsync, AddPartnerAsync, UpdatePartnerAsync, DeletePartnerAsync), а также методы для получения истории продаж и расчета скидки;

– IPartnerTypeService — методы для управления типами партнеров;

– IProductService — методы для управления продукцией;

– ISaleHistoryService — методы для работы с историей продаж;

– IDiscountService — методы для расчета скидок на основе суммы продаж.

2.4.2. Класс PartnerService

Сервис для работы с партнерами содержит следующую логику:

– GetAllPartnersAsync — получает всех партнеров с подключением навигационного свойства PartnerType;

– AddPartnerAsync — добавляет нового партнера с предварительной валидацией (проверка уникальности названия компании, формата email, неотрицательности рейтинга);

– UpdatePartnerAsync — обновляет данные партнера с проверкой существования записи;

– DeletePartnerAsync — удаляет партнера по ID;

– GetPartnerTotalSalesAsync — вычисляет общую сумму продаж партнера через SUM по таблице sales_history;

– GetPartnerDiscountAsync — рассчитывает скидку партнера на основе общей суммы продаж через DiscountService.

```
public async Task<Partner> AddPartnerAsync(Partner partner)
{   if (partner == null)
        throw new ArgumentNullException("partner");
    ValidatePartner(partner);
    var existing = await _context.Partners
        .FirstOrDefaultAsync(p => p.CompanyName == partner.CompanyName);
    if (existing != null)
        throw new InvalidOperationException(
            $"Партнёр с названием '{partner.CompanyName}' уже существует");
    partner.CreatedDate = DateTime.Now;
    partner.ModifiedDate = DateTime.Now;

    _context.Partners.Add(partner);
    await _context.SaveChangesAsync();
    return partner;}
```

Метод ValidatePartner выполняет проверку данных:

– проверяет, что название компании не пустое;

– проверяет, что рейтинг неотрицательный;

– проверяет корректность формата email через EmailAddressAttribute.

2.4.3. Класс DiscountService

Сервис для расчета скидок содержит пороговые значения и соответствующие им проценты:

– менее 10000 рублей — базовый (0%);

– 10000-49999 рублей — бронзовый (5%);

– 50000-299999 рублей — серебряный (10%);

– более 300000 рублей — золотой (15%).

Методы сервиса:

– CalculateDiscount(decimal totalSales) — возвращает коэффициент скидки (от 0.00 до 0.15);

```
public decimal CalculateDiscount(decimal totalSales)
{
    if (totalSales < 0)
        throw new ArgumentException("Сумма продаж не может быть отрицательной",
"totalSales");

    if (totalSales < ThresholdLevel1)
        return DiscountLevel0;   // 0%
    else if (totalSales < ThresholdLevel2)
        return DiscountLevel1;   // 5%
    else if (totalSales < ThresholdLevel3)
        return DiscountLevel2;   // 10%
    else
        return DiscountLevel3;   // 15%
}
```

– GetDiscountPercent(decimal totalSales) — возвращает процент скидки в виде целого числа;

```
public int GetDiscountPercent(decimal totalSales)
 {
     return (int)(CalculateDiscount(totalSales) * 100);
 }
```

– GetDiscountLevel(decimal totalSales) — возвращает текстовое описание уровня скидки.

```
public string GetDiscountLevel(decimal totalSales)
 {
     if (totalSales < ThresholdLevel1)
     {return "Базовый (0%)";}
     else if (totalSales < ThresholdLevel2)
     {return "Бронзовый (5%)";}
     else if (totalSales < ThresholdLevel3)
     {return "Серебряный (10%)";}
     else
     {return "Золотой (15%)";}
 }
```

2.4.4. Класс ProductService

Сервис для работы с продукцией реализует следующие методы:

– GetAllAsync — получает список всех товаров, отсортированный по ID;

– AddAsync — добавляет новый товар с проверкой уникальности артикула и валидацией цены;

– UpdateAsync — обновляет данные товара;

– DeleteAsync — удаляет товар по ID.

При добавлении товара проверяется:

– наличие названия продукции;

– неотрицательность цены;

– уникальность артикула (чтобы не было дублей).

2.4.5. Класс SaleHistoryService

Сервис для работы с историей продаж содержит методы:

– GetAllAsync — получает все записи о продажах с подключением данных о партнере и продукте;

– GetByPartnerIdAsync — получает историю продаж конкретного партнера;

– AddAsync — добавляет новую запись о продаже с проверкой существования партнера и продукта;

– UpdateAsync — обновляет данные о продаже;

– DeleteAsync — удаляет запись о продаже.

При добавлении продажи выполняется проверка:

– количество товара должно быть больше 0;

– сумма сделки должна быть неотрицательной;

– партнер и продукт должны существовать в базе данных.

2.5. Обработка исключений

Для корректной обработки ошибок предметной области я создал пользовательский класс исключения PartnerException. Он наследуется от базового класса Exception и предоставляет три конструктора:

– PartnerException() — конструктор без параметров;

– PartnerException(string message) — конструктор с сообщением об ошибке;

– PartnerException(string message, Exception innerException) — конструктор с сообщением и внутренним исключением.

```
public class PartnerException : Exception
{
    public PartnerException() : base() { }

    public PartnerException(string message) : base(message) { }

    public PartnerException(string message, Exception innerException)
        : base(message, innerException) { }
}
```

Этот класс используется в сервисах для выбрасывания специфичных ошибок, связанных с работой с партнерами (например, при попытке добавить партнера с некорректными данными).

2.6. Внедрение зависимостей

Все сервисы поддерживают внедрение зависимостей через конструктор. Это позволяет легко тестировать код и заменять реализации при необходимости.

Например, PartnerService принимает в конструкторе ApplicationContext и IDiscountService:

```
public PartnerService(ApplicationContext context, IDiscountService discountService)
{
    _context = context ?? throw new ArgumentNullException("context");
    _discountService = discountService ?? throw new ArgumentNullException("discountService");
}
```

Также все сервисы реализуют интерфейс IDisposable для корректного освобождения ресурсов контекста базы данных:

```
public void Dispose()
{   Dispose(true);
    GC.SuppressFinalize(this);}
protected virtual void Dispose(bool disposing)
{
    if (!_disposed)
    {
        if (disposing)
        {
            _context.Dispose();
        }
        _disposed = true;
    }
}
```

2.7 Диаграмма классов

После добавления всех классов в shumilovskih_library была создана диаграмма классов.

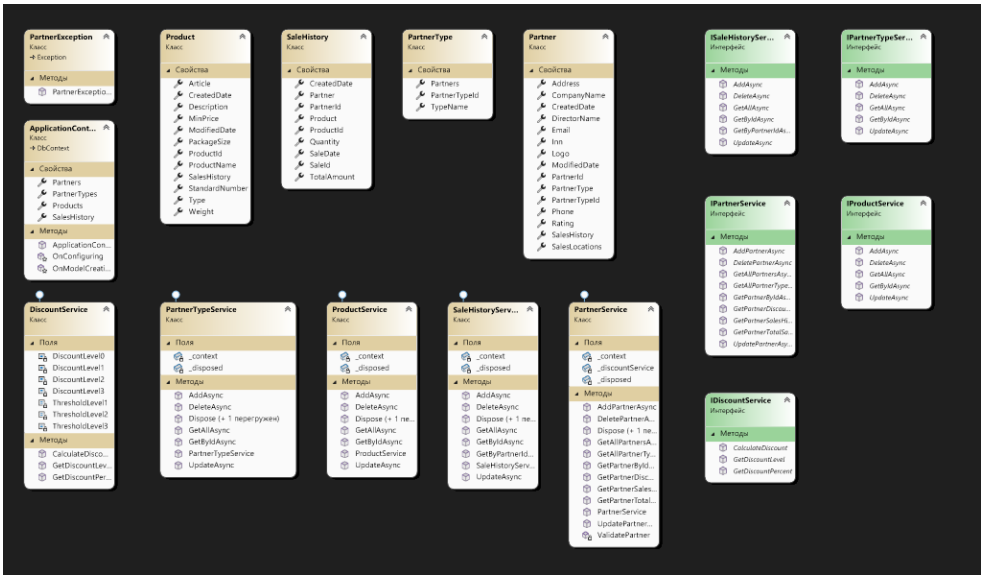

Рис. 7 — **диаграмма классов**

РАЗДЕЛ 3. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ

3.1. Инструменты и среда тестирования

Для обеспечения надежности разработанного программного обеспечения и проверки корректности бизнес-логики мною было выполнено модульное тестирование (Unit Testing). Тестирование проводилось на уровне сервисов библиотеки классов shumilovskih_library, что позволило изолировать логику от пользовательского интерфейса и реальной базы данных.

В качестве инструментов тестирования были использованы:

– фреймворк MSTest– встроенная платформа для тестирования в экосистеме Visual Studio;

– класс LibraryTests – основной класс, содержащий набор тестовых методов для проверки всех сервисов приложения.

3.2. Сценарии тестирования

В ходе работы было разработано более 20 тестовых методов, покрывающих основные функциональные требования проекта. Тесты можно разделить на несколько групп по проверяемым компонентам:

Тестирование сервиса скидок (DiscountService):

– проверка корректности расчета процента скидки в зависимости от суммы продаж (границы 10 000, 50 000, 300 000 рублей).

– проверка обработки некорректных данных (например, выбрасывание исключения при отрицательной сумме продаж).

Тестирование CRUD-операций (Создание, Чтение, Обновление, Удаление):

– партнеры (PartnerService): добавление нового партнера, получение списка, поиск по ID, обновление данных, удаление.

– типы партнеров (PartnerTypeService): проверка уникальности наименования типа при добавлении.

– продукция (ProductService): добавление товара с проверкой уникальности артикула, редактирование цены и веса.

– история продаж (SaleHistoryService): фиксация продажи, привязка к партнеру и товару, удаление записи.

Тестирование бизнес-логики:

– проверка метода GetPartnerTotalSalesAsync – корректность подсчета общей суммы продаж конкретного партнера через агрегацию данных.

– проверка метода GetPartnerDiscountAsync – связка расчета суммы продаж и получения уровня скидки.

– проверка освобождения ресурсов (Dispose) – корректное закрытие контекста базы данных после работы сервиса.

3.3. Результаты тестирования

Все разработанные тестовые сценарии были успешно пройдены. В ходе отладки в окне вывода (Output) фиксировались сообщения о статусе каждого теста.

Основные итоги тестирования:

– логика расчета скидок работает верно на всех пороговых значениях;

– ограничения целостности (уникальность артикула, названия компании) отрабатывают корректно;

– ошибки валидации (отрицательная цена, пустое название) вызывают ожидаемые исключения;

– внедрение зависимостей и работа с контекстом базы данных не приводят к утечкам ресурсов.

Проведенное тестирование подтвердило работоспособность библиотеки классов и готовность программного продукта к интеграции с пользовательским интерфейсом.

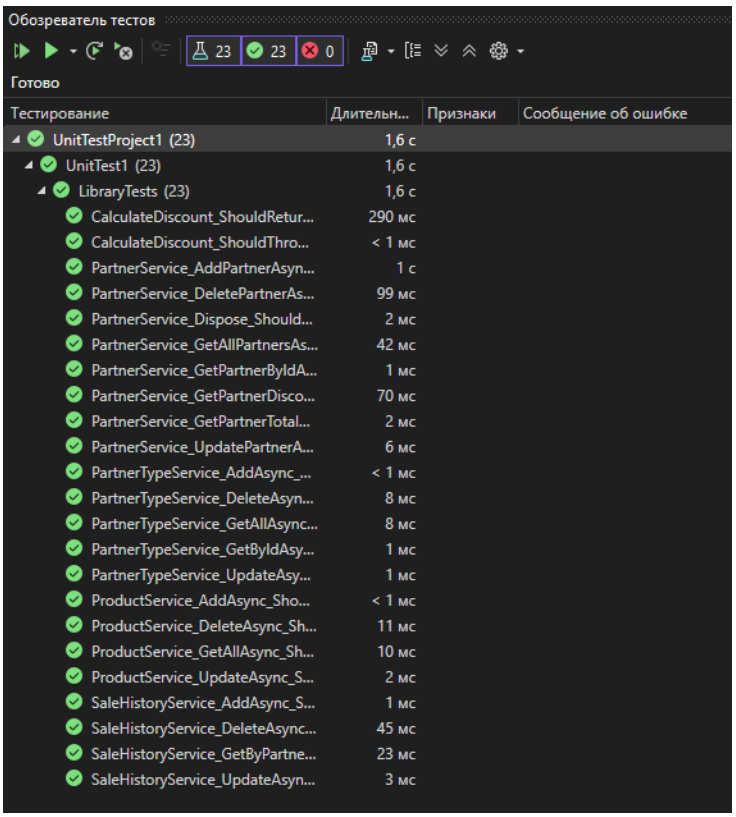

Рис. 8 — **выполненные тесты**

РАЗДЕЛ 4. СОЗДАНИЕ ПРОЕКТА WPF

4.1. Общая структура приложения

Основное приложение проекта я реализовал как десктопную программу на технологии WPF (Windows Presentation Foundation) в проекте shumilovskih_pp. Весь интерфейс выполнен в едином стиле с использованием ресурсов, которые я описал в файле App.xaml. Там я задал цветовую палитру и общие стили для кнопок, панелей и заголовков, чтобы все окна выглядели одинаково.

Основные цвета и стили в App.xaml:

```
<Color x:Key="PrimaryColor">#2C3E50</Color>
<Color x:Key="AccentColor">#3498DB</Color>
<Color x:Key="SuccessColor">#27AE60</Color>
<Color x:Key="DangerColor">#E74C3C</Color>
<Style x:Key="PrimaryButtonStyle" TargetType="Button" BasedOn="{StaticResource
BaseButtonStyle}">
    <Setter Property="Background" Value="{StaticResource AccentBrush}"/>
</Style>
<Style x:Key="DangerButtonStyle" TargetType="Button" BasedOn="{StaticResource
BaseButtonStyle}">
    <Setter Property="Background" Value="{StaticResource DangerBrush}"/>
</Style>
```

4.2. Главное окно MainWindow

4.2.1 Интерфейс окна MainWindow

Главное окно — это первое, что видит пользователь при запуске приложения. Оно служит основной панелью управления, где отображается список всех партнёров слева и история продаж выбранного партнёра справа. Через это окно можно переходить к управлению партнёрами, продукцией и продажами через меню.

Структура интерфейса: интерфейс главного окна я построил на основе Grid с разделением на строки и столбцы. В верхней части расположено меню, а основную рабочую область я разделил на две панели с помощью GridSplitter, чтобы пользователь мог менять их размер перетаскиванием.

Верхнее меню:

```
<Menu Grid.Row="0" Background="{StaticResource PrimaryBrush}">
    <MenuItem Header="_Партнёры">
        <MenuItem Header="Добавить партнера" Click="AddPartner_Click"/>
        <MenuItem Header="Изменить партнера" Click="EditPartner_Click"/>
        <MenuItem Header="Удалить партнера" Click="DeletePartner_Click"/>
    </MenuItem>
    <MenuItem Header="_Продукция">
        <MenuItem Header="Просмотр продукции" Click="ViewProducts_Click"/>
        <MenuItem Header="Добавить продукцию" Click="AddProduct_Click"/>
    </MenuItem>
    <MenuItem Header="_Продажи">
        <MenuItem Header="Просмотр истории" Click="ViewSales_Click"/>
        <MenuItem Header="Добавить продажу" Click="AddSale_Click"/>
    </MenuItem>
</Menu>
```

Меню содержит три основных раздела. Каждый пункт меню привязан к обработчику события Click, который открывает соответствующее окно или выполняет действие.

Левая панель — Список партнёров:

```
<DataGrid x:Name="PartnersDataGrid"
          AutoGenerateColumns="False"
          IsReadOnly="True"
          SelectionChanged="PartnersDataGrid_SelectionChanged">
    <DataGrid.Columns>
        <DataGridTextColumn Header="ID" Binding="{Binding PartnerId}"/>
        <DataGridTextColumn Header="Компания" Binding="{Binding
CompanyName}"/>
        <DataGridTextColumn Header="Тип" Binding="{Binding
PartnerType.TypeName}"/>
        <DataGridTextColumn Header="Рейтинг" Binding="{Binding Rating}"/>
        <DataGridTextColumn Header="Директор" Binding="{Binding
DirectorName}"/>
        <DataGridTextColumn Header="Email" Binding="{Binding Email}"/>
    </DataGrid.Columns>
</DataGrid>
```

Таблица настроена на режим только для чтения (IsReadOnly="True"), чтобы пользователь не мог случайно изменить данные прямо в ячейках. Столбцы я прописал вручную через DataGridTextColumn, указав привязку к

свойствам модели Partner. Важный момент — событие SelectionChanged, которое срабатывает при выборе строки и запускает загрузку истории продаж.

Правая панель — История продаж:

```
<Border Background="#E8F5E9" Padding="10">
    <StackPanel Orientation="Horizontal">
        <TextBlock Text="Общая сумма продаж: " FontWeight="Bold"/>
        <TextBlock x:Name="TotalSalesTextBlock" Text="0 ₽"
                   FontWeight="Bold" Foreground="{StaticResource
SuccessBrush}"/>
        <TextBlock Text="Скидка: " FontWeight="Bold"/>
        <TextBlock x:Name="DiscountTextBlock" Text="0%"
                   FontWeight="Bold" Foreground="{StaticResource
DangerBrush}"/>
    </StackPanel>
</Border>
```

Зелёная панель сверху показывает итоговую сумму и текущую скидку партнёра. Я использовал разные цвета из ресурсов (SuccessBrush для суммы, DangerBrush для скидки), чтобы визуально выделить важную информацию. Ниже расположена ещё одна DataGrid для отображения истории продаж.

Строка состояния:

```
<StatusBar>
    <StatusBarItem>
        <TextBlock x:Name="StatusTextBlock" Text="Готов к работе"/>
    </StatusBarItem>
</StatusBar>
```

Внизу окна я добавил StatusBar для вывода служебных сообщений — например, «Партнер добавлен» или «Ошибка загрузки». Это помогает пользователю понимать, что происходит в приложении.

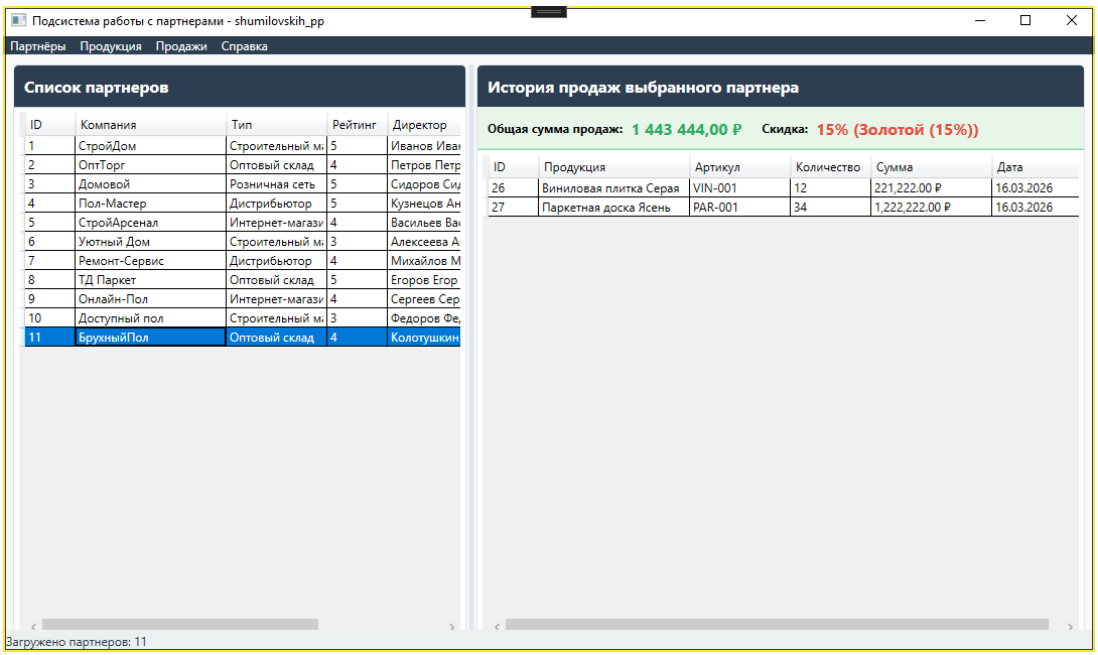

Рис. 9 — **окно MainWindow**

4.2.2 Логика работы окна MainWindow

В коде окна (MainWindow.xaml.cs) я реализовал всю логику взаимодействия с пользователем и базой данных.

Инициализация сервисов:

```
public MainWindow()
{
    InitializeComponent();
    _context = new ApplicationContext();
    _partnerService = new PartnerService(_context, new DiscountService());
    _saleHistoryService = new SaleHistoryService(_context);
    _discountService = new DiscountService();
    _productService = new ProductService(_context);
    _partnerTypeService = new PartnerTypeService(_context);
    LoadPartners();
    UpdateStatus("Приложение запущено.");
}
```

В конструкторе я создаю экземпляры всех сервисов, которые понадобятся для работы. Контекст базы данных передаётся в сервисы, чтобы они могли выполнять запросы. Сразу после инициализации вызывается метод LoadPartners() для загрузки списка партнёров.

Загрузка списка партнёров:

```
private async void LoadPartners()
```

```
        {
            try
            {
                var partners = await _partnerService.GetAllPartnersAsync();
                PartnersDataGrid.ItemsSource = partners;
                UpdateStatus($"Загружено партнеров: {partners.Count}");
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Ошибка загрузки партнеров: {ex.Message}", "Ошибка",
                    MessageBoxButton.OK, MessageBoxImage.Error);
                UpdateStatus("Ошибка загрузки партнеров");
            }
        }
```

Метод асинхронный (async), чтобы интерфейс не зависал во время запроса к базе. Я получаю список через сервис и устанавливаю его в свойство ItemsSource таблицы. Если возникает ошибка — показываю MessageBox и обновляю статус.

Выбор партнёра и загрузка продаж:

```
        private void PartnersDataGrid_SelectionChanged(object sender,
SelectionChangedEventArgs e)
        {
            if (PartnersDataGrid.SelectedItem is Partner partner)
            {
                _selectedPartner = partner;
                LoadPartnerSales(partner);
            }
        }
        private async void LoadPartnerSales(Partner partner)
        {
            if (partner == null)
            {
                SalesDataGrid.ItemsSource = new List<SaleHistory>();
                TotalSalesTextBlock.Text = "0 ₽";
                DiscountTextBlock.Text = "0%";
                return;
            }
            var sales = await
_saleHistoryService.GetByPartnerIdAsync(partner.PartnerId);
```

```
            SalesDataGrid.ItemsSource = sales;
            var totalSales = await
_partnerService.GetPartnerTotalSalesAsync(partner.PartnerId);
            var discount = await
_partnerService.GetPartnerDiscountAsync(partner.PartnerId);
            TotalSalesTextBlock.Text = $"{totalSales:N2} ₽";
            DiscountTextBlock.Text = $"{discount * 100:N0}%
({_discountService.GetDiscountLevel(totalSales)})";
        }
```

Когда пользователь кликает на строку в таблице партнёров, срабатывает SelectionChanged. Я сохраняю выбранного партнёра в переменную _selectedPartner и вызываю загрузку его продаж. В методе LoadPartnerSales я запрашиваю историю продаж, считаю общую сумму через сервис и рассчитываю скидку. Результаты выводятся в текстовые блоки с форматированием (например, N2 для двух знаков после запятой).

Обработчики меню:

```
        private void AddPartner_Click(object sender, RoutedEventArgs e)
        {
            var window = new PartnersWindow(_partnerService, _partnerTypeService, null);
            window.Owner = this;
            if (window.ShowDialog() == true)
            {
                LoadPartners();
                UpdateStatus("Партнер добавлен");
            }
        }

        private async void DeletePartner_Click(object sender, RoutedEventArgs e)
        {
            if (_selectedPartner == null)
            {
                MessageBox.Show("Выберите партнера для удаления", "Внимание",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }
            var result = MessageBox.Show(
                $"Вы действительно хотите удалить партнера
\"{_selectedPartner.CompanyName}\"?",
```

```
            "Подтверждение удаления", MessageBoxButton.YesNo,
MessageBoxImage.Question);
        if (result == MessageBoxResult.Yes)
        {
            await _partnerService.DeletePartnerAsync(_selectedPartner.PartnerId);
            LoadPartners();
            LoadPartnerSales(null);
            UpdateStatus("Партнер удален");
        }
    }
```

При добавлении партнёра я открываю окно PartnersWindow в модальном режиме (ShowDialog). Если окно вернуло true (данные сохранены), я перезагружаю список. При удалении сначала проверяю, выбран ли партнёр, затем запрашиваю подтверждение через MessageBox и только после этого удаляю запись через сервис.

Обновление статуса:

```
private void UpdateStatus(string message)
{
    StatusTextBlock.Text = message;
}
```

Простой метод для вывода сообщений в строку состояния. Я использую его после каждой операции, чтобы пользователь видел результат своих действий.

4.3. Окно управления партнерами PartnersWindow

4.3.1 Интерфейс PartnersWindow

Окно PartnersWindow предназначено для добавления новых партнеров в базу данных или редактирования информации об уже существующих. Оно открывается как модальное диалоговое окно из главного меню или контекстного действия в MainWindow. После успешного сохранения данных окно возвращает результат true, что сигнализирует главному окну о необходимости обновить список партнеров.

Структура интерфейса: интерфейс окна я построил на основе Grid с вертикальной прокруткой (ScrollViewer), так как полей для заполнения

достаточно много. Все элементы управления стилизованы через ресурсы App.xaml для единого визуального стиля.

Основные поля ввода:

```
<ScrollViewer Grid.Row="1" VerticalScrollBarVisibility="Auto">
    <StackPanel>
        <!-- Наименование компании (обязательное) -->
        <TextBlock Text="Наименование компании *" FontWeight="SemiBold"/>
        <TextBox x:Name="CompanyNameTextBox" Height="30" Margin="0,0,0,15"/>

        <!-- Тип партнера (выпадающий список с возможностью ввода нового) -->
        <TextBlock Text="Тип партнера *" FontWeight="SemiBold"/>
        <ComboBox x:Name="PartnerTypeComboBox" Height="30"
                  IsEditable="True" TextSearch.TextPath="TypeName"/>

        <!-- Рейтинг (только цифры) -->
        <TextBlock Text="Рейтинг *" FontWeight="SemiBold"/>
        <TextBox x:Name="RatingTextBox" Height="30"
                 PreviewTextInput="NumberValidationTextBox"/>

        <!-- Остальные поля: ИНН, Адрес, Директор, Телефон, Email -->
        <!-- ... -->

        <!-- Логотип (текстовое поле + кнопка обзора) -->
        <TextBlock Text="Логотип (путь к файлу)"/>
        <Grid Margin="0,0,0,15">
            <TextBox x:Name="LogoTextBox" Height="30" Grid.Column="0"/>
            <Button             Grid.Column="1"             Content="Обзор..."
Click="BrowseLogo_Click"/>
        </Grid>
    </StackPanel>
</ScrollViewer>
```

В нижней части окна расположены кнопки «Сохранить» и «Отмена». Поля, помеченные звездочкой, являются обязательными для заполнения. Для поля «Тип партнера» я установил свойство IsEditable="True", чтобы пользователь мог не только выбрать существующий тип из списка, но и ввести новый прямо в поле.

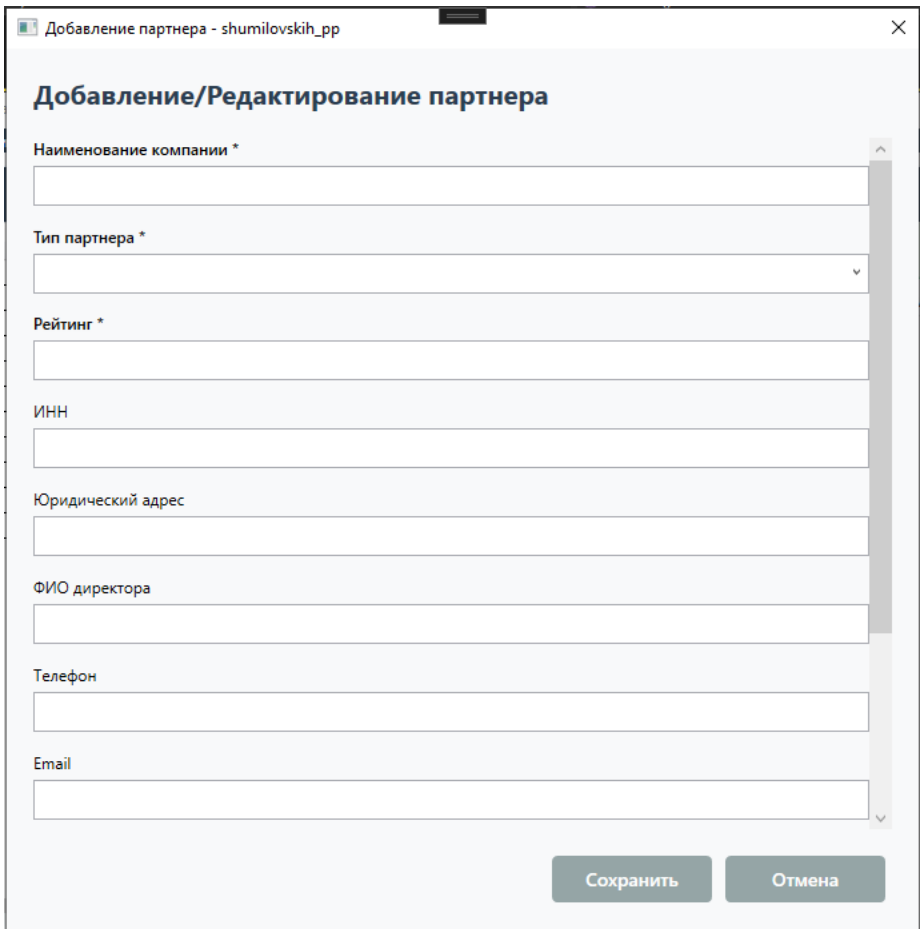

Рис. 10 — **окно PartnersWindow**

4.3.2 Логика работы окна PartnersWindow

В коде окна (PartnersWindow.xaml.cs) я реализовал логику различения режимов работы (добавление или редактирование), валидацию данных и взаимодействие с сервисами.

Инициализация и режимы работы:

```
public PartnersWindow(IPartnerService partnerService,
                      IPartnerTypeService partnerTypeService,
                      Partner partnerToEdit = null)
{
    InitializeComponent();
    _partnerService = partnerService;
    _partnerTypeService = partnerTypeService;
    _selectedPartner = partnerToEdit;
    _isEditMode = partnerToEdit != null; // Определяем режим

    if (_isEditMode)
    {
        Title = "Редактирование партнера - shumilovskih_pp";
```

```
            LoadPartnerData(); // Заполняем поля данными существующего партнера
        }
        else
        {
            Title = "Добавление партнера - shumilovskih_pp";
        }
        this.Loaded += async (s, e) => await LoadPartnerTypesAsync();
    }
```

В конструкторе я проверяю, передан ли объект партнера. Если передан — окно работает в режиме редактирования, заголовок меняется, и поля заполняются данными. Если нет — режим добавления. Также я подписываюсь на событие Loaded, чтобы асинхронно загрузить список типов партнеров сразу после открытия окна.

Загрузка типов партнеров:

```
    private async System.Threading.Tasks.Task LoadPartnerTypesAsync()
    {
        try
        {
            var partnerTypes = await _partnerTypeService.GetAllAsync();
            PartnerTypeComboBox.ItemsSource = partnerTypes;
            PartnerTypeComboBox.DisplayMemberPath = "TypeName";
            PartnerTypeComboBox.SelectedValuePath = "PartnerTypeId";

            if (_isEditMode && _selectedPartner != null)
            {
                PartnerTypeComboBox.SelectedValue = _selectedPartner.PartnerTypeId;
            }
        }
        catch (Exception ex)
        {

            MessageBox.Show($"Ошибка загрузки типов: {ex.Message}", "Ошибка",
                MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
```

Метод загружает все доступные типы из базы данных и привязывает их к ComboBox. Я указал DisplayMemberPath и SelectedValuePath, чтобы в списке отображалось название типа, а сохранялся его идентификатор.

Валидация ввода чисел:

Для поля «Рейтинг» я запретил ввод букв, обрабатывая событие PreviewTextInput. Это предотвращает ошибки типа данных еще на уровне интерфейса.

```
private void NumberValidationTextBox(object sender, TextCompositionEventArgs e)
{
    Regex regex = new Regex("[^0-9]+"); // Разрешаем только цифры
    e.Handled = regex.IsMatch(e.Text);
}
```

Выбор логотипа:

Кнопка «Обзор...» открывает стандартный диалог выбора файла. Путь к выбранному файлу записывается в текстовое поле.

```
private void BrowseLogo_Click(object sender, RoutedEventArgs e)
{
    var openFileDialog = new OpenFileDialog
    {
        Filter = "Image files (*.jpg;*.jpeg;*.png)|*.jpg;*.jpeg;*.png|All files
(*.*)|*.*",
        Title = "Выберите файл логотипа"
    };
    if (openFileDialog.ShowDialog() == true)
    {
        LogoTextBox.Text = openFileDialog.FileName;
    }
}
```

Сохранение данных — самая сложная часть логики находится в обработчике кнопки «Сохранить». Здесь я проверяю обязательные поля, обрабатываю создание нового типа партнера (если пользователь ввел его вручную) и вызываю соответствующий метод сервиса.

```
private async void Save_Click(object sender, RoutedEventArgs e)
{
    try
    {
        // 1. Проверка обязательных полей
        if (string.IsNullOrWhiteSpace(CompanyNameTextBox.Text))
        {
```

```
                MessageBox.Show("Наименование компании обязательно", "Ошибка
валидации",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }
            // 2. Обработка типа партнера (существующий или новый)
            int partnerTypeId;
            if (PartnerTypeComboBox.SelectedItem == null)
            {
                string newTypeName = PartnerTypeComboBox.Text.Trim();
                // Ищем существующий тип или создаем новый
                var existingType = (await _partnerTypeService.GetAllAsync())
                    .FirstOrDefault(pt => pt.TypeName.ToLower() ==
newTypeName.ToLower());
                if (existingType != null)
                {
                    partnerTypeId = existingType.PartnerTypeId;
                }
                else
                {
                    var newPartnerType = new PartnerType { TypeName = newTypeName
};
                    var createdType = await
_partnerTypeService.AddAsync(newPartnerType);
                    partnerTypeId = createdType.PartnerTypeId;
                }
            }
            else
            {
                partnerTypeId = (int)PartnerTypeComboBox.SelectedValue;
            }

            // 3. Проверка рейтинга
            if (!int.TryParse(RatingTextBox.Text, out int rating) || rating < 0)
            {
                MessageBox.Show("Рейтинг должен быть неотрицательным числом",
"Ошибка валидации",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }
```

```
            // 4. Сохранение в зависимости от режима
            if (_isEditMode && _selectedPartner != null)
            {
                // Обновление существующего
                _selectedPartner.CompanyName = CompanyNameTextBox.Text.Trim();
                _selectedPartner.PartnerTypeId = partnerTypeId;
                _selectedPartner.Rating = rating;
                // ... присваивание остальных полей ...
                _selectedPartner.ModifiedDate = DateTime.Now;
                await _partnerService.UpdatePartnerAsync(_selectedPartner);
            }
            else
            {
                // Создание нового
                var partner = new Partner
                {
                    CompanyName = CompanyNameTextBox.Text.Trim(),
                    PartnerTypeId = partnerTypeId,
                    Rating = rating,
                    // ... заполнение остальных полей ...
                    CreatedDate = DateTime.Now,
                    ModifiedDate = DateTime.Now
                };
                await _partnerService.AddPartnerAsync(partner);
            }

            DialogResult = true; // Сообщаем главному окну об успехе
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка",
                MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
```

В этом методе я реализовал важную особенность: если пользователь ввел тип партнера вручную, а такого типа нет в базе, система автоматически создает его перед сохранением партнера. Это упрощает работу пользователя — не нужно заранее идти в справочник типов.

В конце метода я устанавливаю DialogResult = true. Это стандартный механизм WPF для диалоговых окон: главное окно проверяет это значение после закрытия PartnersWindow и понимает, что нужно перезагрузить список партнеров.

4.4 Окно управления продукцией ProductsWindow

4.4.1. Интерфейс ProductsWindow

Окно ProductsWindow предназначено для просмотра всего каталога товаров, доступных в системе. Через это окно пользователь может добавить новую позицию, изменить характеристики существующего товара или удалить его из базы. Оно открывается через главное меню программы и работает как отдельное диалоговое окно.

Структура интерфейса: интерфейс окна я построил на основе Grid с тремя строками: заголовок, таблица данных и панель кнопок. Основное место занимает DataGrid, который отображает список товаров.

```
<DataGrid Grid.Row="1" x:Name="ProductsDataGrid"
          AutoGenerateColumns="False" IsReadOnly="True"
          SelectionChanged="ProductsDataGrid_SelectionChanged">
    <DataGrid.Columns>
        <DataGridTextColumn    Header="ID"    Binding="{Binding    ProductId}"
Width="50"/>
        <DataGridTextColumn       Header="Наименование"       Binding="{Binding
ProductName}" Width="150"/>
        <DataGridTextColumn   Header="Артикул"   Binding="{Binding   Article}"
Width="100"/>
        <DataGridTextColumn    Header="Цена"    Binding="{Binding    MinPrice,
StringFormat={}{0:N2} ₽}" Width="80"/>
        <!-- Остальные колонки -->
    </DataGrid.Columns>
</DataGrid>
```

В нижней части окна расположена панель с кнопками управления. Я вынес их в отдельный StackPanel, чтобы они были всегда видны пользователю.

```
<StackPanel Grid.Row="2" Orientation="Horizontal" HorizontalAlignment="Left">
    <Button Content="Добавить" Width="100" Click="AddProduct_Click"/>
    <Button Content="Изменить" Width="100" Click="EditProduct_Click"/>
    <Button Content="Удалить" Width="100" Click="DeleteProduct_Click"/>
```

```
</StackPanel>
<Button Grid.Column="1" Content="Закрыть" Width="100" Click="Close_Click"/>
```

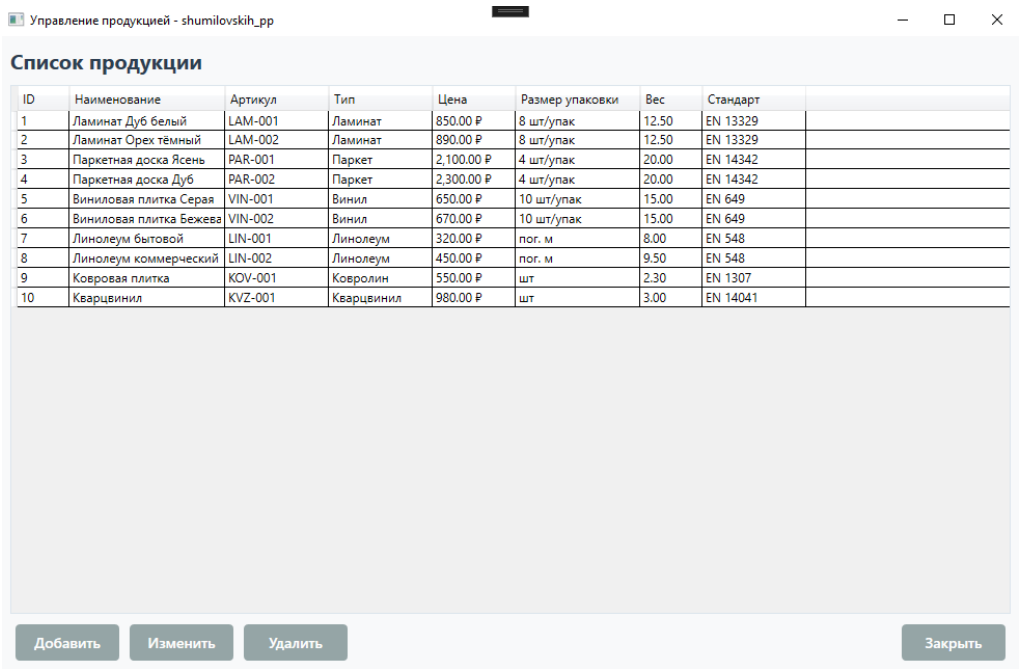

Рис. 11 — **интерфейс ProductsWindow**

4.4.2 Логика работы окна ProductsWindow

В коде окна ProductsWindow.xaml.cs я реализовал загрузку данных при старте и обработку кнопок управления.

Загрузка списка товаров:

При открытии окна вызывается метод LoadProducts, который асинхронно запрашивает данные через сервис и привязывает их к таблице.

```
private async Task LoadProducts()
{
    try
    {
        var products = await _productService.GetAllAsync();
        ProductsDataGrid.ItemsSource = products;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка загрузки продукции: {ex.Message}", "Ошибка",
            MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

Удаление товара:

Перед удалением я обязательно проверяю, выбрана ли строка в таблице, и запрашиваю подтверждение у пользователя. Это защищает от случайного удаления данных.

```
private async void DeleteProduct_Click(object sender, RoutedEventArgs e)
{
    if (_selectedProduct == null)
    {
        MessageBox.Show("Выберите продукцию для удаления", "Внимание",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }
    // Запрос подтверждения
    var result = MessageBox.Show(
        $"Вы действительно хотите удалить продукцию \"{_selectedProduct.ProductName}\"?",
        "Подтверждение удаления", MessageBoxButton.YesNo, MessageBoxImage.Question);

    if (result == MessageBoxResult.Yes)
    {
        await _productService.DeleteAsync(_selectedProduct.ProductId);
        await LoadProducts(); // Обновляем таблицу после удаления
    }
}
```

Если товар участвует в продажах, база данных не позволит удалить его из-за ограничения внешнего ключа (ON DELETE RESTRICT), и сервис выбросит исключение, которое я перехватываю и показываю пользователю.

4.5 Окно добавления и изменения продукции ProductEditWindow

4.5.1 Интерфейс ProductEditWindow

Окно ProductEditWindow используется для создания новой карточки товара или изменения данных существующего. Оно открывается в модальном режиме из окна списка продукции.

Структура интерфейса: форма содержит поля для ввода основных характеристик товара. Я сгруппировал их в ScrollViewer, чтобы окно помещалось на экране даже при большом количестве полей.

```
<ScrollViewer Grid.Row="1" VerticalScrollBarVisibility="Auto">
```

```
        <StackPanel>
            <TextBlock Text="Наименование *" FontWeight="SemiBold"/>
            <TextBox x:Name="ProductNameTextBox" Height="30"/>

            <TextBlock Text="Артикул *" FontWeight="SemiBold"/>
            <TextBox x:Name="ArticleTextBox" Height="30"/>

            <TextBlock Text="Мин. цена *" FontWeight="SemiBold"/>
            <TextBox x:Name="MinPriceTextBox" Height="30"
                     PreviewTextInput="NumberValidationTextBox"/>
            <!-- Остальные поля: Вес, Описание, Стандарт -->
        </StackPanel>
    </ScrollViewer>
```

Для полей «Цена» и «Вес» я добавил обработку события PreviewTextInput, чтобы разрешить ввод только цифр и разделителей дроби.

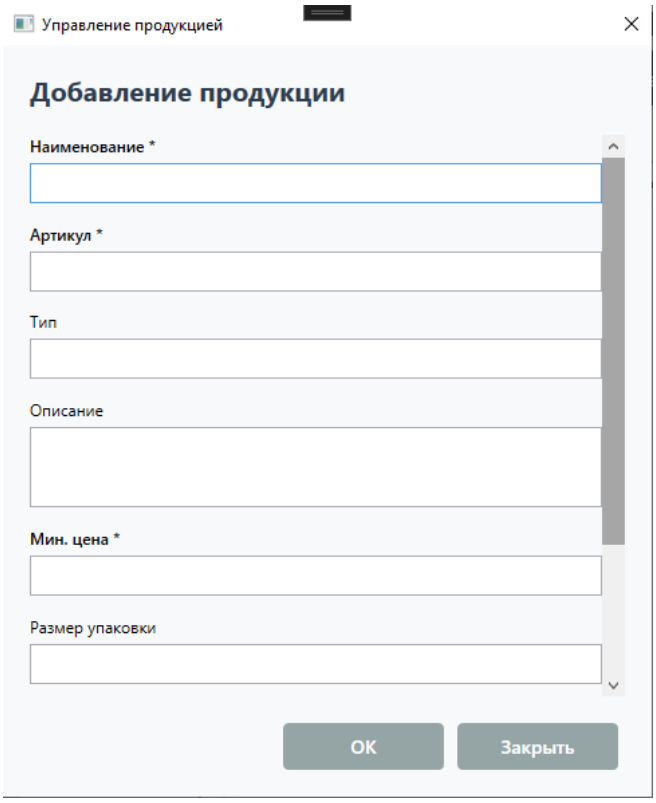

Рис. 12 — **интерфейс ProductEditWindow**

4.5.2 Логика работы окна ProductEditWindow

В коде окна я реализовал проверку данных перед сохранением и различие режимов (добавление или редактирование).

Валидация ввода:

Метод NumberValidationTextBox использует регулярное выражение для фильтрации ввода. Это предотвращает ошибки формата данных еще на этапе ввода.

```
private void NumberValidationTextBox(object sender, TextCompositionEventArgs e)
{
    // Разрешаем только цифры, точку и запятую
    Regex regex = new Regex("[^0-9.,]+");
    e.Handled = regex.IsMatch(e.Text);
}
```

Сохранение данных:

В обработчике кнопки «ОК» я проверяю обязательные поля и корректность числовых значений. Если товар редактируется, я обновляю существующий объект, если нет — создаю новый.

```
private async void Save_Click(object sender, RoutedEventArgs e)
{
    try
    {
        // Проверка обязательных полей
        if (string.IsNullOrWhiteSpace(ProductNameTextBox.Text))
        {
            MessageBox.Show("Наименование продукции обязательно", "Ошибка",
                MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }
        // Проверка цены
        if (!decimal.TryParse(MinPriceTextBox.Text, out decimal minPrice) ||
minPrice < 0)
        {
            MessageBox.Show("Цена должна быть неотрицательным числом",
"Ошибка",
                MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }

        if (_isEditMode && _selectedProduct != null)
        {
            // Обновление существующего товара
            _selectedProduct.ProductName = ProductNameTextBox.Text.Trim();
```

```
                _selectedProduct.MinPrice = minPrice;
                _selectedProduct.ModifiedDate = DateTime.Now;
                await _productService.UpdateAsync(_selectedProduct);
            }
            else
            {
                // Создание нового товара
                var product = new Product
                {
                    ProductName = ProductNameTextBox.Text.Trim(),
                    MinPrice = minPrice,
                    CreatedDate = DateTime.Now,
                    ModifiedDate = DateTime.Now
                };
                await _productService.AddAsync(product);
            }
            DialogResult = true; // Сообщаем об успехе
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка",
                MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
```

Особенностью логики является проверка уникальности артикула. Она реализована на уровне сервиса, но в окне я обрабатываю исключение, если товар с таким артикулом уже существует, и сообщаю об этом пользователю.

4.6 Окно истории продаж SalesHistoryWindow

4.6.1 Интерфейс SalesHistoryWindow

Окно SalesHistoryWindow предназначено для просмотра всей истории продаж конкретного партнёра. Оно открывается из главного окна после выбора партнёра в списке или через меню «Продажи». В верхней части окна отображается информация о партнёре, общая сумма всех его покупок и рассчитанная скидка. Ниже расположена таблица с детальными записями о каждой продаже.

Структура интерфейса: интерфейс окна я построил на основе Grid с четырьмя строками: заголовок, информационная панель, таблица данных и панель кнопок.

Заголовок и информационная панель:

```
<TextBlock Grid.Row="0" Text="История продаж партнера"
           FontSize="20" FontWeight="Bold"
           Foreground="{StaticResource PrimaryBrush}" Margin="0,0,0,10"/>

<Border Grid.Row="1" Background="#E8F5E9" Padding="10" Margin="0,0,0,10"
        BorderBrush="{StaticResource SuccessBrush}" BorderThickness="1"
CornerRadius="4">
    <StackPanel>
        <TextBlock x:Name="PartnerNameTextBlock" Text="Партнер: "
                   FontWeight="Bold" FontSize="14"/>
        <StackPanel Orientation="Horizontal" Margin="0,5,0,0">
            <TextBlock Text="Общая сумма продаж: " FontWeight="Bold"/>
            <TextBlock x:Name="TotalSalesTextBlock" Text="0 ₽"
                       FontWeight="Bold" Foreground="{StaticResource
SuccessBrush}"/>
            <TextBlock Text="Скидка: " FontWeight="Bold"/>
            <TextBlock x:Name="DiscountTextBlock" Text="0%"
                       FontWeight="Bold" Foreground="{StaticResource
DangerBrush}"/>
        </StackPanel>
    </StackPanel>
</Border>
```

Зелёная панель (Background="#E8F5E9") визуально выделяет важную финансовую информацию. Я использовал разные цвета для суммы и скидки, чтобы пользователь сразу видел ключевые показатели.

Таблица продаж:

```
<DataGrid Grid.Row="2" x:Name="SalesDataGrid"
          AutoGenerateColumns="False" IsReadOnly="True"
          SelectionMode="Single" SelectionUnit="FullRow"
          SelectionChanged="SalesDataGrid_SelectionChanged">
    <DataGrid.Columns>
        <DataGridTextColumn Header="ID" Binding="{Binding SaleId}" Width="50"/>
        <DataGridTextColumn        Header="Продукция"        Binding="{Binding
Product.ProductName}" Width="150"/>
```

```
            <DataGridTextColumn Header="Артикул" Binding="{Binding Product.Article}" Width="100"/>
            <DataGridTextColumn Header="Количество" Binding="{Binding Quantity}" Width="80"/>
            <DataGridTextColumn Header="Сумма" Binding="{Binding TotalAmount, StringFormat={}{0:N2} ₽}" Width="120"/>
            <DataGridTextColumn Header="Дата" Binding="{Binding SaleDate, StringFormat={}{0:dd.MM.yyyy}}" Width="100"/>
        </DataGrid.Columns>
    </DataGrid>
```

Таблица настроена на режим только для чтения. Я вручную прописал колонки с привязкой к свойствам модели SaleHistory. Форматирование суммы (StringFormat={}{0:N2} ₽) добавляет знак рубля и два знака после запятой.

Панель кнопок:

```
<StackPanel Grid.Row="3" Orientation="Horizontal" HorizontalAlignment="Left">
    <Button Content="Добавить" Width="130" Click="AddSale_Click"/>
    <Button Content="Изменить" Width="100" Click="EditSale_Click"/>
    <Button Content="Удалить" Width="100" Click="DeleteSale_Click"/>
</StackPanel>
<Button Grid.Column="1" Content="Закрыть" Width="100" Click="Close_Click"/>
```

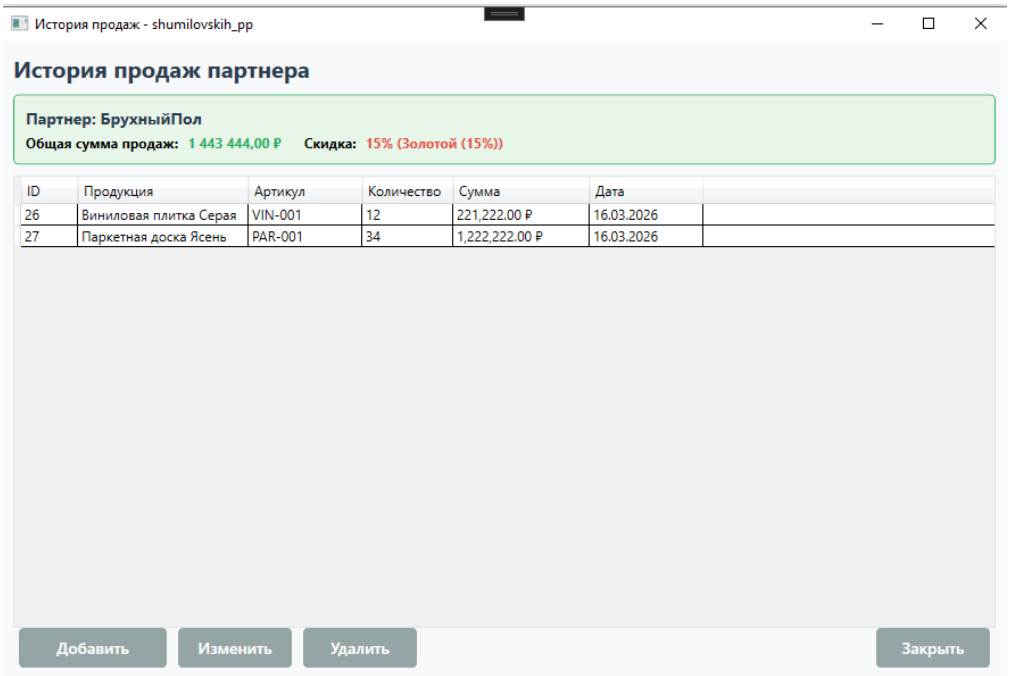

Рис. 13 — **интерфейс SalesHistoryWindow**

4.6.2 Логика работы окна SalesHistoryWindow

В коде окна (SalesHistoryWindow.xaml.cs) я реализовал загрузку данных, расчёт скидки и обработку кнопок управления.

Инициализация и загрузка данных:

```
public SalesHistoryWindow(ISaleHistoryService saleHistoryService, IProductService productService, Partner partner)
{
    InitializeComponent();
    _saleHistoryService = saleHistoryService;
    _productService = productService;
    _discountService = new DiscountService();
    _partner = partner;
    PartnerNameTextBlock.Text = $"Партнер: {partner.CompanyName}";
    LoadSales();
}
```

В конструкторе я принимаю объект партнёра, чтобы знать, для кого загружать историю. Сразу после инициализации вызывается метод LoadSales().

Расчёт суммы и скидки:

```
private async Task LoadSales()
{
    try
    {
        var sales = await _saleHistoryService.GetByPartnerIdAsync(_partner.PartnerId);
        SalesDataGrid.ItemsSource = sales;
        // Считаем общую сумму всех продаж
        var totalSales = sales.Sum(s => s.TotalAmount);
        var discount = _discountService.CalculateDiscount(totalSales);
        // Выводим результаты
        TotalSalesTextBlock.Text = $"{totalSales:N2} ₽";
        DiscountTextBlock.Text = $"{discount * 100:N0}% ({_discountService.GetDiscountLevel(totalSales)})";
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка загрузки истории продаж: {ex.Message}", "Ошибка",
            MessageBoxButton.OK, MessageBoxImage.Error);
    }
```

```
}
```

В этом методе я загружаю все продажи партнёра через сервис, затем считаю общую сумму через LINQ-метод Sum(). После этого запрашиваю размер скидки у DiscountService и вывожу её в текстовый блок вместе с текстовым описанием уровня (Базовый, Бронзовый и т.д.).

Добавление новой продажи:

```
private async void AddSale_Click(object sender, RoutedEventArgs e)
{
    var window = new SaleEditWindow(_saleHistoryService, _productService, _partner);
    window.Owner = this;
    if (window.ShowDialog() == true)
    {
        await LoadSales(); // Обновляем историю после добавления
    }
}
```

При нажатии кнопки «Добавить» открывается окно SaleEditWindow. Если продажа успешно сохранена (окно вернуло true), я перезагружаю историю, чтобы отобразить новую запись.

Удаление продажи:

```
private async void DeleteSale_Click(object sender, RoutedEventArgs e)
{
    if (_selectedSale == null)
    {
        MessageBox.Show("Выберите продажу для удаления", "Внимание",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }
    var result = MessageBox.Show("Вы действительно хотите удалить запись о продаже?",
        "Подтверждение удаления", MessageBoxButton.YesNo, MessageBoxImage.Question);
    if (result == MessageBoxResult.Yes)
    {
        await _saleHistoryService.DeleteAsync(_selectedSale.SaleId);
        await LoadSales();
    }
}
```

Перед удалением я проверяю, выбрана ли запись в таблице, и запрашиваю подтверждение. Это защищает от случайного удаления данных.

4.7 Окно добавления продажи SaleEditWindow

4.7.1 Интерфейс SaleEditWindow

Окно SaleEditWindow используется для регистрации новой продажи или редактирования существующей записи. Оно открывается в модальном режиме из окна истории продаж. Пользователь выбирает товар, указывает количество и сумму сделки, а также дату продажи.

Структура интерфейса: интерфейс окна представляет собой форму с полями ввода, размещёнными в StackPanel.

Основные поля:

```
<TextBlock x:Name="PartnerNameTextBlock" Text="Партнер: "
           FontWeight="Bold" FontSize="14"
           Foreground="{StaticResource PrimaryBrush}" Margin="0,0,0,15"/>
<TextBlock Text="Продукция *" Margin="0,0,0,5" FontWeight="SemiBold"/>
<ComboBox x:Name="ProductComboBox" Height="30" Margin="0,0,0,15"
          DisplayMemberPath="ProductName" SelectedValuePath="ProductId"/>
<TextBlock Text="Количество *" Margin="0,0,0,5" FontWeight="SemiBold"/>
<TextBox x:Name="QuantityTextBox" Height="30" Margin="0,0,0,15" Padding="5"
         PreviewTextInput="NumberValidationTextBox"/>
<TextBlock Text="Сумма *" Margin="0,0,0,5" FontWeight="SemiBold"/>
<TextBox x:Name="TotalAmountTextBox" Height="30" Margin="0,0,0,15" Padding="5"
         PreviewTextInput="NumberValidationTextBox"/>
<TextBlock Text="Дата продажи *" Margin="0,0,0,5" FontWeight="SemiBold"/>
<DatePicker x:Name="SaleDatePicker" Height="30" Margin="0,0,0,15"/>
```

Для полей «Количество» и «Сумма» я добавил обработку события PreviewTextInput, чтобы разрешить ввод только цифр и разделителей дроби. Поле «Партнёр» заполняется автоматически из переданного объекта и доступно только для чтения.

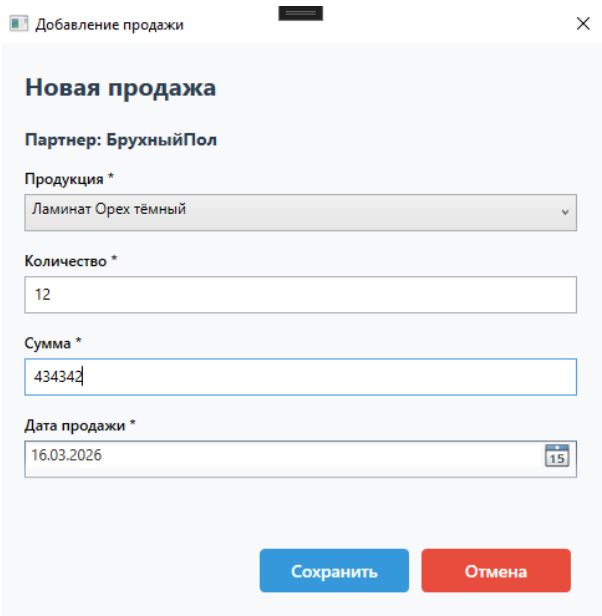

Рис. 14 — **интерфейс SaleEditWindow**

4.7.2 Логика работы окна SaleEditWindow

В коде окна (SaleEditWindow.xaml.cs) я реализовал загрузку списка товаров, валидацию данных и сохранение продажи.

Инициализация и загрузка товаров:

```
public SaleEditWindow(ISaleHistoryService saleHistoryService, IProductService productService, Partner partner)
{
    InitializeComponent();
    _saleHistoryService = saleHistoryService;
    _productService = productService;
    _partner = partner;
    PartnerNameTextBlock.Text = $"Партнер: {partner.CompanyName}";
    SaleDatePicker.SelectedDate = DateTime.Today;
    LoadProducts();
}

private async void LoadProducts()
{
    try
    {
        var products = await _productService.GetAllAsync();
        ProductComboBox.ItemsSource = products;
    }
    catch (Exception ex)
    {
```

```
            MessageBox.Show($"Ошибка загрузки продукции: {ex.Message}", "Ошибка",
                MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
```

При открытии окна я сразу загружаю все товары из базы данных и привязываю их к ComboBox. Дата продажи по умолчанию устанавливается в текущую.

Валидация ввода чисел:

```
    private void NumberValidationTextBox(object sender, TextCompositionEventArgs
e)
    {
        Regex regex = new Regex("[^0-9.,]+");
        e.Handled = regex.IsMatch(e.Text);
    }
```

Этот метод обрабатывает событие PreviewTextInput для полей количества и суммы. Регулярное выражение запрещает ввод букв и специальных символов, кроме цифр, точки и запятой.

Сохранение продажи:

```
    private async void Save_Click(object sender, RoutedEventArgs e)
    {
        try
        {
            // Проверка выбора товара
            if (ProductComboBox.SelectedItem == null)
            {
                MessageBox.Show("Выберите продукцию", "Ошибка",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }

            // Проверка количества
            if (!int.TryParse(QuantityTextBox.Text, out int quantity) || quantity
<= 0)
            {
                MessageBox.Show("Количество должно быть больше 0", "Ошибка",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }
```

```
            // Проверка суммы
            if (!decimal.TryParse(TotalAmountTextBox.Text, out decimal
totalAmount) || totalAmount < 0)
            {
                MessageBox.Show("Сумма должна быть неотрицательной", "Ошибка",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }

            // Проверка даты
            if (!SaleDatePicker.SelectedDate.HasValue)
            {
                MessageBox.Show("Выберите дату продажи", "Ошибка",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }
            // Создание новой записи
            var sale = new SaleHistory
            {
                PartnerId = _partner.PartnerId,
                ProductId = (int)ProductComboBox.SelectedValue,
                Quantity = quantity,
                TotalAmount = totalAmount,
                SaleDate = SaleDatePicker.SelectedDate.Value,
                CreatedDate = DateTime.Now
            };
            await _saleHistoryService.AddAsync(sale);

            MessageBox.Show("Продажа добавлена", "Информация",
                MessageBoxButton.OK, MessageBoxImage.Information);
            DialogResult = true;
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка",
                MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
```

В этом методе я последовательно проверяю все обязательные поля. Если какое-то поле не прошло валидацию, показываю сообщение и прерываю

выполнение. Только после успешной проверки всех данных создаю объект SaleHistory и отправляю его в сервис для сохранения. В конце устанавливаю DialogResult = true, чтобы окно истории знало о успешном добавлении.

ЗАКЛЮЧЕНИЕ

В ходе производственной практики мною была разработана информационная система «Подсистема работы с партнерами» (проект shumilovskih_pp). Программный продукт реализован на языке C# с использованием технологии WPF для пользовательского интерфейса и СУБД PostgreSQL 16 для хранения данных.

Основные результаты работы:

1. Спроектирована база данных с четырьмя таблицами (partners, products, sales_history, partner_types), настроены внешние ключи и ограничения целостности.

2. Создана библиотека классов с многоуровневой архитектурой (Models, Context, Services), инкапсулирующая бизнес-логику и доступ к данным через Entity Framework Core.

3. Разработано desktop-приложение с функционалом CRUD для управления партнерами, продукцией и историей продаж.

4. Реализована автоматическая система скидок, рассчитывающая уровень лояльности партнера на основе накопленной суммы покупок.

5. Выполнено модульное тестирование сервисов.

В результате удалось автоматизировать процесс учета контрагентов и транзакций, минимизировать риски ошибок ручного ввода и обеспечить прозрачность взаимоотношений с партнерами. В ходе практики были закреплены навыки проектирования БД, разработки на .NET, работы с WPF и тестирования программного обеспечения. Все поставленные цели и задачи практики были успешно выполнены.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. https://metanit.com/sql/postgresql/

2.https://213.230.96.51:8090/files/ebooks/dasturlash/Postgresql%20osnovy.pdf

3. https://www.oslogic.ru/knowledge/598/shpargalka-po-osnovnym-komandam-postgresql/

4. https://postgrespro.ru/education/books/sqlprimer

5. https://metanit.com/sharp/tutorial/

6. https://metanit.com/sharp/wpf/

7. https://metanit.com/sharp/efcore/

8.https://codernet.ru/articles/drugoe/modulnoe_testirovanie_yunittestirovanie_chto_eto_tipyi_instrumentyi/

9. https://learn.microsoft.com/ru-ru/visualstudio/test/walkthrough-creating-and-running-unit-testsfor-managed-code?view=vs-2022